WEEK 1 · LESSON 1 OF 24 · HANDOUT

The Deployment Gap: From Notebook to Product

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=wRnKcWIZEvk — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2020: Painless Machine Learning in Production — https://www.youtube.com/watch?v=8Zn4hV9xE1I

🎓 **Go deeper:** MIT 6.S191 (2024): Building AI Models in the Wild — https://www.youtube.com/watch?v=ZAGiinWiFsE

Your goal in this lesson: Explain why a model that works in a notebook is not yet a product, name the parts of a real machine-learning system, and save a model with the information needed to use it safely later.

Welcome to the capstone course

In the earlier courses you built models that predict, classify and generate. In this course you learn to **deliver** them: put a model behind an API, test it, ship it, watch it in production, check it is fair, protect people's data, document it — and finish with your own end-to-end capstone project.

All course long we use one realistic story: a savings and credit cooperative (SACCO) wants a model that predicts whether a loan applicant will **repay**. Loans change lives, so mistakes and unfairness matter. The data is made up — no real people.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

The notebook model

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

An AUC of about **0.88** is a good model. In a notebook, the job looks done. It is not.

What changes in production

In the notebook	In production
You run the code once	It answers requests all day, every day
Clean, known data	Missing fields, typing mistakes, new kinds of applicants
You are the only user	Loan officers and applicants rely on the answer
Errors show up as red text	Errors are silent: a wrong number looks like a right number
The world is frozen	Prices, incomes and behaviour change every month
No rules	Data protection law, fairness, appeals, audits

A famous Google paper, "Hidden Technical Debt in Machine Learning Systems" (2015), showed that in real systems the model code is only a **small part**. Around it you need data collection and checks, feature code, serving, monitoring, configuration and process.

The parts of an ML system

1. **Data pipeline** — collects and checks the data.
2. **Training pipeline** — turns data into a model in a repeatable way.
3. **Model artifact** — the saved model file plus its information (version, features, metrics).
4. **Serving** — an API or batch job that makes predictions.
5. **Monitoring** — watches inputs, outputs, speed, errors and quality.
6. **Feedback** — real outcomes come back and trigger retraining.
7. **People and rules** — owners, reviewers, documentation, appeals, law.

Save the model with its "passport"

A model file alone is dangerous: later nobody knows which features it expects, how good it was, or which data trained it. Save a small metadata file next to it:

```
import datetime
import hashlib
import json

import joblib
import sklearn

joblib.dump(model, "loan_model_v1.joblib")

data_hash = hashlib.sha256(pd.util.hash_pandas_object(train,
index=False).values.tobytes()).hexdigest()[:16]
meta = {
    "name": "loan-repayment",
    "version": "1.0.0",
    "trained_on": datetime.date.today().isoformat(),
    "features": FEATURES,
    "target": TARGET,
    "threshold": 0.5,
    "metrics": {"auc": round(roc_auc_score(test[TARGET], proba), 3),
                "accuracy": round(accuracy_score(test[TARGET], proba >= 0.5), 3)},
    "training_rows": len(train),
    "data_fingerprint": data_hash,
    "sklearn_version": sklearn.__version__,
}
with open("loan_model_v1.json", "w") as f:
    json.dump(meta, f, indent=2)
print(json.dumps(meta, indent=2))
```

The **data fingerprint** proves which data trained the model. The **sklearn version** matters because a model saved with one version may not load in another.

A silent production bug

Load the model the way a server would, and send one applicant:

```
loaded = joblib.load("loan_model_v1.joblib")
info = json.load(open("loan_model_v1.json"))

def predict_one(applicant):
    row = pd.DataFrame([applicant])[info["features"]]
    p = float(loaded.predict_proba(row)[0, 1])
    return {"repay_probability": round(p, 3), "approve": p >= info["threshold"]}

good = {"income_rwf": 350_000, "loan_rwf": 200_000, "years_member": 5,
        "late_payments": 0, "salaried": 1, "momo_tx_month": 40}
print("correct units:   ", predict_one(good))

mistake = dict(good, income_rwf=350)          # a form that asks for income "in thousands"
print("income in 1,000s:", predict_one(mistake))
```

The second answer is badly wrong — but the code raised **no error**. This is called **training–serving skew**: the data at serving time does not look like the training data. Real systems stop this with input checks:

```
LIMITS = {"income_rwf": (10_000, 20_000_000), "loan_rwf": (10_000, 5_000_000),
          "years_member": (0, 60), "late_payments": (0, 50), "salaried": (0, 1),
"momo_tx_month": (0, 1000)}

def check_input(applicant):
    problems = []
    for name, (low, high) in LIMITS.items():
        value = applicant.get(name)
        if value is None:
            problems.append(f"{name} is missing")
        elif not low <= value <= high:
            problems.append(f"{name}={value} is outside {low}..{high}")
    return problems

print(check_input(good))
print(check_input(mistake))
```

Rejecting a strange request with a clear message is far better than returning a confident wrong answer.

Key terms

- **Deployment** — making a model available to real users or systems.
- **Model artifact** — the saved model plus its metadata.
- **Metadata** — information about the model: version, features, metrics, data, library versions.
- **Training–serving skew** — serving data that differs from training data, causing silent errors.
- **Input validation** — checking requests before predicting.

Remember this: A notebook model becomes a product only with everything around it: versioned artifacts with metadata, input checks, serving, monitoring, people and rules.

Name: _____ Date: _____

Activity

Task 1

Send an applicant with `late_payments` missing to `predict_one`. What happens? Then try `check_input`.

Task 2

Add a check that the loan is not more than 20 times the income.

Task 3

Change the version to `1.0.1`, save again, and list everything that should change in the metadata.

Task 4

Draw the seven parts of an ML system for a problem in your own organisation.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why is a good notebook model not yet a product?
   - A. Notebooks cannot train models
   - B. Production needs serving, checks, monitoring, people and rules around the model
   - C. The AUC is too low
   - D. Products never use Python
2. What should be saved next to the model file?
   - A. Nothing
   - B. Only the accuracy
   - C. The whole internet
   - D. Metadata: version, features, metrics, data fingerprint and library versions
3. What is training–serving skew?
   - A. A slow server
   - B. A fairness metric
   - C. Serving data that differs from training data, causing silent errors
   - D. A type of chart
4. Why are production errors often dangerous?
   - A. They delete the model
   - B. They are silent: a wrong number looks like a right number
   - C. They are loud
   - D. They only happen at night
5. What should the API do with an income of 350 RWF for a salaried worker?
   - A. Predict anyway
   - B. Reject it with a clear message
   - C. Ignore the field
   - D. Double it
6. Why record the scikit-learn version?
   - A. It is required by law
   - B. A model saved with one version may not load in another
   - C. For marketing
   - D. It sets the threshold

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.